AI INTERACTION LOG – Sprint 2

Entry 1: Backend API & Database Migration Implementation

Task ID/Title: Sprint 2 Job Application Submission and Tracking API (002_applications.sql & server.js)

Purpose of AI Use: Coding, requirements analysis, and database schema design

Chat Link or Prompt/Response:

Prompt:

"Help me implement the Sprint 2 backend deliverables for Job Application Submission and Tracking in CareerConnect (002_applications.sql and server.js). We need endpoints to submit applications (POST /api/applications), view and filter applications (GET /api/applications/me), and update application statuses (PATCH /api/applications/:id/status) across the 5 mandatory states (Applied, Under Review, Interview, Offered, Rejected), while preventing duplicate applications to the same job."

AI Response:

"To implement Sprint 2 Job Application Submission and Tracking:

1. **Database Migration (db/migrations/002_applications.sql):** Create the applications table with id, user_id (foreign key referencing users(id)), job_id, job_title, company, cover_letter, status (defaulting to 'Applied'), and created_at/updated_at timestamps. Add UNIQUE (user_id, job_id) to prevent duplicate submissions at the database level.
2. **Submit Application (POST /api/applications):** Validate required fields (jobId, jobTitle, company, and optional coverLetter length limits). Insert the record with status 'Applied' and return 201 Created. Catch PostgreSQL error code 23505 (unique constraint violation) and return 409 Conflict with { message: 'You have already applied to this job.' }.
3. **Retrieve & Filter Applications (GET /api/applications/me):** Query applications belonging to req.user.id ordered by created_at DESC. Support optional query parameter ?status= to filter results dynamically after validating against allowed statuses.
4. **Update Lifecycle Status (PATCH /api/applications/:id/status):** Validate the incoming status against ['Applied', 'Under Review', 'Interview', 'Offered',

'Rejected'] (returning 400 Bad Request if invalid), verify ownership (user_id = req.user.id), and update the record."

AI-Suggested Content: Generated the PostgreSQL migration script (002_applications.sql) defining the applications table with foreign key references to users(id) and a compound unique constraint on (user_id, job_id). Provided Express.js route handlers in server.js for submitting applications (POST /api/applications), retrieving and filtering user applications by status (GET /api/applications/me), and updating application statuses (PATCH /api/applications/:id/status) across the five required lifecycle states (Applied, Under Review, Interview, Offered, Rejected).

Validation: Conducted manual code review against Sprint 2 acceptance criteria, verified SQL constraint behavior against PostgreSQL error code 23505 (409 Conflict on duplicate submissions), and executed automated integration tests in test/database.test.js using pg-mem.

Decision: Modified before use

Reflection: The AI assistance accelerated boilerplate SQL and Express route setup while ensuring edge cases--such as invalid status strings (400 Bad Request), unauthorized access (401 Unauthorized), and duplicate job applications (409 Conflict)--were properly handled.

Responsible Person: Jack Wu

Entry 2: Git Merge Conflict Resolution & JWT Authentication Integration

Task ID/Title: Resolve Merge Conflicts in server.js and test/database.test.js with Teammate Auth & Resume PRs

Purpose of AI Use: Debugging, code integration, and review

Chat Link or Prompt/Response:

Prompt:

"My GitHub Pull Request (feature/sprint2-application-tracking-jack) has merge conflicts in server.js and test/database.test.js after my teammate merged JWT authentication (authenticateUser) and resume upload routes into main. How do I resolve these conflicts so both features and test suites work together?"

AI Response:

"To resolve the merge conflicts cleanly without losing either feature:

1. **In server.js:** Remove the Git conflict markers (<<<<<<<, =======, >>>>>>>). Keep the teammate's bcrypt/jsonwebtoken login route (/api/auth/login), authenticateUser JWT middleware, and multer/jszip resume endpoints (/api/resumes). Update your Sprint 2 application routes (POST /api/applications, GET /api/applications/me, PATCH /api/applications/:id/status) to use authenticateUser middleware and read the authenticated user ID from req.user.userId (or req.user.id).
2. **In test/database.test.js:** Combine both test blocks inside the main integration test runner so that the resume upload/ownership/size-limit tests execute first, followed by user login token generation and the job application submission, duplicate conflict (409), status update (PATCH), and filtering (GET) assertions."

AI-Suggested Content: Analyzed Git conflict markers (<<<<<<<, =======, >>>>>>>) between the local feature/sprint2-application-tracking-jack branch and the updated main branch containing teammate implementations for JWT authentication (authenticateUser) and resume uploads (multer, jszip). Provided a unified merge structure that preserved teammate resume/auth routes while upgrading the job application endpoints to use the new JWT Bearer token middleware.

Validation: Verified through GitHub Pull Request CI/CD checks (confirming zero merge conflicts and passing status checks) and ran npm test locally after running git pull to

ensure both teammate resume tests and new job application persistence tests passed together (pass 1, fail 0).

Decision: Accepted

Reflection: Resolving overlapping changes in server.js and test/database.test.js highlighted how shared integration files create merge bottlenecks. Using AI to cross-check conflict blocks prevented accidental deletion of teammate resume validation tests while properly securing the new Sprint 2 endpoints with JWT tokens.

Responsible Person: Jack Wu

Entry 3: Automated Test Suite & VS Code REST Client Configuration

Task ID/Title: Local API Verification and Alternative REST Client Setup (test.http)

Purpose of AI Use: Testing, troubleshooting, and tool configuration

Chat Link or Prompt/Response:

Prompt:

"Thunder Client isn't working because it requires an account login. How can I test my endpoints locally using REST Client instead? Also, when I send a request I get {\"message\": \"Invalid or expired authentication token.\"} and HTTP/1.1 409 Conflict {\"message\": \"User with this email already exists.\"}--how do I fix this and demo it to the professor?"

AI Response:

"Use the lightweight **REST Client** extension in VS Code with a test.http file inside your repository:

HTTP

@baseUrl = http://localhost:3000

@authToken = PASTE_YOUR_JWT_TOKEN_HERE

1. Register User

POST {{baseUrl}}/api/auth/register

Content-Type: application/json

{ \"email\": \"restclient@concordia.ca\", \"password\": \"Password123!\" }

2. Login User

POST {{baseUrl}}/api/auth/login

Content-Type: application/json

{ \"email\": \"restclient@concordia.ca\", \"password\": \"Password123!\" }

3. Submit Job Application

POST {{baseUrl}}/api/applications

Authorization: Bearer {{authToken}}

Content-Type: application/json

{ \"jobId\": \"JOB-101\", \"jobTitle\": \"Software Intern\", \"company\": \"Concordia Tech\", \"coverLetter\": \"Interested in the role.\" }

- **Fixing Invalid or expired authentication token:** Run the Login request (POST /api/auth/login), copy the token string (eyJhbGciOi...) from the JSON response, and paste it into @authToken = ... at the top of test.http.
- **Understanding 409 Conflict:** That indicates the user email (or job application) is already saved in the in-memory database, proving duplicate prevention works. Skip directly to the Login step or increment the email (restclient2@concordia.ca) if registering a new account."

AI-Suggested Content: After encountering account login restrictions with the Thunder Client GUI extension, the AI recommended the VS Code REST Client extension and generated a structured test.http script. The script chained user registration (POST /api/auth/register), JWT login token generation (POST /api/auth/login), and authenticated requests (Authorization: Bearer {{authToken}}) for application creation, status updates, and query filtering.

Validation: Executed npm test in PowerShell to verify automated assertions (pass 1, fail 0) and interactively ran each HTTP request block in test.http inside VS Code, inspecting the live JSON payloads and HTTP status codes (201 Created, 200 OK, and 409 Conflict).

Decision: Modified before use

Reflection: Switching from Thunder Client to a repository-native test.http file removed external GUI dependencies and provided a repeatable, version-controlled demonstration script for live sprint evaluation.

Responsible Person: Jack Wu